1. Real-Time Face Recognition for Campus Security

■ Constraints Identified From the Brief

- Hard: recognize only registered students/staff — unregistered faces must never be force-matched.
- Hard: must run at real-time speed on inexpensive (non-GPU) hardware.
- Hard: must keep working when entrance lighting is poor, uneven, or backlit.
- Optimization goal: maximize correct-match rate for enrolled individuals without breaking the above.

■ Search for a Valid Design (Trial → Conflict → Revision)

1) Initial trial assignment: use a deep-learning face detector at full camera resolution (1280x720) on every frame, paired with a high-capacity embedding network. This maximizes accuracy on paper.
2) Conflict check: at full resolution and full-frame-rate DNN inference, per-frame processing time on a low-cost CPU exceeds the real-time budget — the 'low-cost hardware, real-time' constraint is violated.
3) Revision 1: reduce frame resolution before detection and restrict full detection to every k-th frame, filling intermediate frames with a lightweight tracker (KCF). Re-check: latency now fits the real-time budget.
4) Second conflict check: at reduced resolution under poor lighting, the detector begins missing faces in shadowed areas — the lighting-robustness constraint is now violated by Revision 1.
5) Revision 2: insert a CLAHE-based local-contrast enhancement step before detection, restoring visibility of faces in shadow/backlight without requiring higher resolution — lighting constraint re-satisfied without reopening the speed conflict.
6) Third conflict check: with all faces now detected, a plain 'closest match wins' recognizer occasionally assigns an enrolled identity to a visitor who is not registered — the no-false-match constraint is violated.
7) Revision 3: replace the plain closest-match rule with a thresholded distance check — any face whose best match distance exceeds a calibrated cut-off is labelled 'Unknown' instead of assigned an identity. All three constraints now hold simultaneously — search terminates at this configuration.

■ Final Constraint-Satisfying Pipeline

1) Capture frames at reduced resolution via cv2.VideoCapture; apply CLAHE and a light Gaussian blur.
2) Run a DNN or Haar face detector every k frames; track detected faces between detection cycles with KCF.
3) Align each face (eye-line horizontal) and encode it with a pretrained embedding network.
4) Compare the embedding to the enrolled database using distance matching with a calibrated rejection threshold.
5) Log/display the identity (or 'Unknown') in real time.

■ Constraint Satisfaction Checklist

Constraint	How It Is Satisfied	Verification / Check
Real-time on low-cost hardware	Reduced resolution + periodic detection + tracker fill-in	Measured per-frame latency stays under target frame interval
Robust under poor/uneven lighting	CLAHE contrast normalization before detection	Detector recall checked on shadow/backlit sample frames
No false match for unregistered faces	Thresholded distance rejection ('Unknown' label)	Distance threshold tuned against a held-out unregistered-face sample

2. Intelligent Traffic Violation Detection System

■ Constraints Identified From the Brief

- Hard: every vehicle must be counted exactly once as it crosses the line.
- Hard: shadows, sensor noise, and swaying background clutter must not be counted as vehicles.
- Hard: overlapping/close vehicles must not be merged into one count or double-counted.
- Optimization goal: correctly flag abnormal movement (wrong-way, speeding, signal violation) using the same tracked data.

■ Search for a Valid Design (Trial → Conflict → Revision)

1) Initial trial assignment: detect foreground blobs each frame using simple frame differencing, and increment the count whenever a blob overlaps the counting line in that frame.
2) Conflict check: a vehicle moving slowly near the line is counted many times across consecutive frames while it overlaps the line — the 'exactly once' constraint is violated.
3) Revision 1: assign each blob a persistent track ID (centroid tracking) and only increment the count the first time that specific ID's centroid crosses the line, using a per-ID 'already counted' flag.
4) Second conflict check: frame differencing also flags moving shadows and swaying roadside branches as blobs, which then get track IDs and get counted as vehicles — the 'no shadows/clutter' constraint is violated.
5) Revision 2: replace frame differencing with MOG2 background subtraction with shadow detection enabled, and add a minimum-area/aspect-ratio filter on candidate blobs — shadows and small clutter are now excluded before tracking even begins.
6) Third conflict check: when two vehicles pass close together, the centroid tracker occasionally swaps their IDs or merges them into one track — the 'no merge/double-count' constraint is violated.
7) Revision 3: upgrade the tracker to a Kalman-filter-based method (SORT-style) that predicts each vehicle's expected position and uses that prediction to resolve close-proximity ambiguity rather than relying on centroid distance alone. All three constraints now hold — search terminates.

■ Final Constraint-Satisfying Pipeline

1) Capture and crop to the road ROI; apply grayscale conversion and Gaussian blur.
2) Apply MOG2 background subtraction with shadow detection; clean the mask with morphological opening then closing.
3) Filter contours by minimum area and aspect ratio to retain only vehicle-plausible blobs.
4) Track surviving blobs with a Kalman-filter-based tracker, assigning a persistent ID to each vehicle.

5) Increment the count once per ID at the moment of line-crossing; compute direction/speed from the same track for violation checks.

■ Constraint Satisfaction Checklist

Constraint	How It Is Satisfied	Verification / Check
Exactly-once counting	Per-track-ID 'already counted' flag set at line-crossing	Manual count vs system count compared on sample footage
No shadow/clutter false counts	MOG2 shadow flag + area/aspect-ratio filtering	Blob inspection under known-shadow and windy-foliage clips
No ID merge/swap on close vehicles	Kalman-filter position prediction resolves proximity ambiguity	ID-consistency check across frames of close-following vehicles

3. Automated Document Processing System

■ Constraints Identified From the Brief

- Hard: output image must be OCR-ready (clean, high-contrast) despite blur, noise, and uneven illumination in the input.
- Hard: text must be upright/aligned before OCR, regardless of scan skew.
- Hard: the same pipeline must run unattended across a large batch, with no per-document manual settings.
- Optimization goal: maximize OCR text-extraction accuracy across mixed font sizes and layouts.

■ Search for a Valid Design (Trial → Conflict → Revision)

1) Initial trial assignment: convert each scan to grayscale and apply a single global Otsu threshold, then feed the whole page directly into the OCR engine.
2) Conflict check: on scans with uneven lighting (a shadow band down one side), the single global threshold turns the darker half of the page fully black or fully white — the OCR-ready-output constraint is violated for those documents.
3) Revision 1: replace global Otsu thresholding with adaptive thresholding (locally computed per neighbourhood), which adjusts to local brightness instead of one page-wide cut-off — the illumination problem is resolved without any manual per-file adjustment.
4) Second conflict check: some scanned pages are slightly rotated (skewed) from the scanner feed; OCR accuracy on these specific pages drops sharply even though thresholding is now correct — the alignment constraint is violated.
5) Revision 2: add an automatic skew-angle estimation step (via Hough line transform or minAreaRect on text contours) and rotate each page to horizontal before OCR — applied identically to every document, preserving full automation.
6) Third conflict check: pages with large headings mixed with small body text produce inconsistent OCR line-segmentation when the whole page is processed as one block — accuracy on mixed-font pages remains poor.
7) Revision 3: insert a segmentation stage (contour + projection-profile based) that isolates text blocks/lines before OCR, so each region is read at its own appropriate scale. All constraints now hold — search terminates.

■ Final Constraint-Satisfying Pipeline

1) Batch-load each scanned page; convert to grayscale and denoise with median blur / Non-Local Means.
2) Apply CLAHE for local contrast, then adaptive thresholding for binarization.
3) Apply morphological opening/closing to repair broken strokes and remove residual speckle.

4) Estimate and correct skew angle via rotation so text lines are horizontal.

5) Segment the page into text regions/lines, then run OCR (Tesseract) on each region and reassemble the output text.

■ Constraint Satisfaction Checklist

Constraint	How It Is Satisfied	Verification / Check
OCR-ready output despite uneven lighting/noise	Denoising + CLAHE + adaptive thresholding	Contrast/binarization checked on shadow-band sample scans
Correct alignment despite skew	Automatic skew estimation and rotation	Skew angle re-measured as ~0° after correction
Fully automated across large batches	Fixed-parameter operations with no per-file tuning	Batch run over sample set with no manual intervention

4. Real-Time Multi-Person Tracking and Attendance System

■ Constraints Identified From the Brief

- Hard: each registered student must be marked present exactly once per class session.
- Hard: identity must not be lost due to brief occlusion or head movement.
- Hard: the system must keep up with multiple simultaneous faces with minimal delay.
- Optimization goal: maximize correct-recognition rate for all students detected.

■ Search for a Valid Design (Trial → Conflict → Revision)

1) Initial trial assignment: run face detection and recognition on every face in every frame, and mark a student present the moment their face is recognized.
2) Conflict check: a student visible for the whole class period gets marked present repeatedly, once per frame they're recognized in — the 'exactly once' constraint is violated, and duplicate log entries pile up.
3) Revision 1: introduce a per-student database flag; once set to 'Present', ignore further recognitions of that student for the rest of the session. This resolves duplicate marking.
4) Second conflict check: when a student briefly turns their head or is blocked by a classmate for a couple of frames, the system treats their reappearance as a new, unrecognized face; without persistent tracking the system cannot tell 'reappeared same student' from 'a different student', undermining the flag logic itself.
5) Revision 2: add a persistent tracker (Kalman-filter based) so each physical person keeps one track ID across brief disappearances (tolerating a small number of missed frames via motion prediction), and bind the per-student flag to the track ID rather than to a raw per-frame detection — this makes Revision 1's flag reliable under occlusion.
6) Third conflict check: running full recognition on every face in every frame, now combined with tracking, is still computationally heavy once the classroom has many students — real-time delay creeps up as class size grows.
7) Revision 3: restrict full recognition to only new or not-yet-identified tracks; once a track is bound to a student ID, subsequent frames rely on the (cheap) tracker alone. All constraints now hold together — search terminates.

■ Final Constraint-Satisfying Pipeline

1) Detect all faces per frame using a DNN multi-face detector.
2) Assign/update a persistent track ID per face using a Kalman-filter-based multi-object tracker.
3) Run recognition (embedding + threshold match) only for new/unconfirmed tracks; bind the result to the track ID.
4) Set the corresponding student's 'Present' flag once, and skip re-marking for the rest of the session.
5) Allow tracks to survive a limited number of missed frames (occlusion/head-turn) before being retired.

■ Constraint Satisfaction Checklist

Constraint	How It Is Satisfied	Verification / Check
Exactly-once marking per student	Per-student session flag, set once and never reset	Attendance log checked for duplicate entries per student
Identity retained through occlusion/movement	Kalman-filter tracking with missed-frame tolerance	Track ID continuity verified across a simulated head-turn clip
Low delay with multiple simultaneous faces	Recognition limited to new/unconfirmed tracks only	Per-frame processing time measured as class size increases

5. Smart Home Surveillance and Intrusion Detection

■ Constraints Identified From the Brief

- Hard: shadows and small ambient background changes must not trigger a false alert.
- Hard: an actual unauthorized entry into the defined zone must always be alerted.
- Hard: the system must run continuously on limited (home-grade) computing hardware.
- Optimization goal: minimize false-alarm frequency without weakening real-intrusion sensitivity.

■ Search for a Valid Design (Trial → Conflict → Revision)

1) Initial trial assignment: use simple frame differencing against a fixed reference background frame, and alert immediately whenever any foreground pixels appear inside the monitored zone.
2) Conflict check: moving shadows and gently swaying tree branches near the entrance regularly appear as foreground and repeatedly trigger alerts — the 'no false alert from shadows/background change' constraint is violated.
3) Revision 1: replace static frame differencing with an adaptive background subtractor (MOG2/KNN) that continuously updates its model, and enable its built-in shadow-detection flag to separately label and discard shadow pixels.
4) Second conflict check: even after Revision 1, small leftover noise blobs (leaf-sized) and occasional flicker still cross the zone boundary and cause an alert on a single frame — the false-alarm objective is still not adequately met.
5) Revision 2: add a minimum-area/aspect-ratio filter on candidate blobs (rejecting anything too small or oddly shaped to be a person), plus a multi-frame optical-flow check that requires consistent directional motion before a blob is treated as 'real' motion, since foliage motion is directionally inconsistent.
6) Third conflict check: a genuinely present intruder is occasionally missed because a single momentarily-still frame fails the motion-consistency check, risking under-alerting — the 'must always alert on a real intrusion' constraint is at risk.
7) Revision 3: switch the alert trigger from a per-frame decision to a dwell-time rule — track the confirmed object and alert once it has remained inside the zone for a short minimum duration, rather than requiring every single frame to independently qualify. All constraints now hold together — search terminates.

■ Final Constraint-Satisfying Pipeline

1) Continuously capture from the fixed camera; apply Gaussian blur to reduce sensor noise.
2) Model the background using MOG2/KNN with shadow detection and a slow adaptive learning rate.

3) Clean the foreground mask with morphological opening then closing.

4) Filter candidate contours by area/aspect ratio, and confirm motion with multi-frame optical-flow direction consistency.

5) Track the confirmed object and test its position against the restricted zone polygon; alert once dwell time exceeds a minimum threshold.

■ Constraint Satisfaction Checklist

Constraint	How It Is Satisfied	Verification / Check
No false alert from shadows	MOG2 shadow-detection flag excludes shadow pixels	Verified on footage with strong cast shadows
No false alert from foliage/background drift	Adaptive learning rate + area filter + optical-flow consistency	Verified on windy-foliage sample clips
Reliable alert on genuine intrusion	Dwell-time-based trigger on tracked, zone-confined object	Verified on simulated entry sequences